

Projektowanie i planowanie aplikacji mobilnych

*Od pomysłu do wdrożenia. Profesjonalny proces,
zarządzanie zespołem i obowiązki lidera projektu*

2026

Informatyka

dr inż. Mateusz Pomianek

Faza ideacji i badania rynku

Skład i role zespołu

Agile / Scrum / Kanban

UX/UI i roadmapa produktu

Release & App Store Launch

Metryki i ciągłe doskonalenie

1 Kontekst rynkowy

Dlaczego planowanie jest kluczowe? Statystyki porażek

2 Faza odkrywania

Ideacja, badania rynku, walidacja pomysłu

3 Skład zespołu

Role, odpowiedzialności, modele współpracy

4 Metodyki pracy

Scrum, Kanban, SAFe – wybór i konfiguracja

5 Planowanie techniczne

Architektura, tech stack, roadmapa produktu

6 Proces UX/UI

Design sprint, prototypowanie, testy użyteczności

7 Development

Sprinty, code review, CI/CD, zarządzanie długiem technicznym

8 Lider zespołu

Obowiązki, kompetencje, techniki zarządzania

9 QA i testowanie

Strategie, automatyzacja, definicja Done

10 Launch i po launchu

App Store, metryki, iteracja, zarządzanie kryzysem

Kontekst rynkowy

Dlaczego planowanie jest kluczowe? Statystyki porażek i sukcesów

42%**projektów mobilnych**

upada z powodu braku potrzeby rynkowej

68%**projektów IT**

przekracza budżet lub harmonogram

77%**użytkowników**

porzuca aplikację w ciągu pierwszych 3 dni

0.5%**aplikacji**

osiąga finansowy sukces (Gartner, 2024)

Główne przyczyny niepowodzeń (wg Standish Group CHAOS Report)

Brak zaangażowania użytkowników w proces

**13.1%**

Niekompletne wymagania

**12.4%**

Zmieniające się wymagania i zakres

**11.8%**

Brak wsparcia kierownictwa wyższego szczebla

**7.5%**

Brak kompetencji technicznych w zespole

**7.0%**

Ciekawostka: Średni koszt nieudanego projektu mobilnego dla MŚP wynosi \$240 000 (Clutch, 2024). Prawidłowe planowanie redukuje ryzyko o 35%.

Faza odkrywania

Ideacja, badania rynku i walidacja pomysłu przed napisaniem pierwszej linii kodu.

1 Problem Statement

Zdefiniuj problem, nie rozwiązanie. Użyj formatu: Dla [persona], która ma [problem], nasza aplikacja [rozwiązanie] daje [korzyść] w odróżnieniu od [alternatywy].

2 Market Research

TAM / SAM / SOM. Analiza konkurencji (App Annie, SensorTower). Minimum 20 wywiadów z potencjalnymi użytkownikami przed projektem.

3 Walidacja hipotez

Build-Measure-Learn. Landing page z CTA + Google Ads = test gotowości rynku za ~500 zł. Cel: >5% konwersja.

4 Business Model Canvas

9 bloków: segmenty, propozycja wartości, kanały, relacje, przychody, zasoby, aktywności, partnerzy, koszty.

Framework analizy konkurencji

Kryterium	Twoja app	Konkurent A	Konkurent B
Czas do wartości (TTFV)	< 60s	3 min	5 min
Monetyzacja	Freemium	Paid	Ads
Offline support	Tak	Nie	Częściowy
Ocena App Store	—	3.8★	4.1★

Definicja MVP: Minimum Viable Product

Must Have (MoSCoW)

Funkcje bez których produkt nie ma sensu. Maks. 3-5 dla pierwszego releaseu.

Could Have

Nice-to-have. Backlog priorytet B.

Should Have

Ważne, ale można wydać bez. Dodawane w sprint 2-3.

Won't Have (v1)

Explicite wykluczone z releaseu 1.0. Zarządzanie oczekiwaniami.

Spotify wypuścił MVP z jedną funkcją (streaming muzyki), bez playlist, offline, czy social. Instagram startował wyłącznie jako apka do filtrowania zdjęć (bez stories, reels, zakupów).

Skład i role zespołu

Kto jest potrzebny i za co odpowiada. Modele współpracy i struktury organizacyjne.

Product Owner (PO)

Właściciel backlogu. Priorytety feature'ów. Jedyna osoba która może powiedzieć NIE. Odpowiada za ROI produktu. Czas: 100% projektu.

Tech Lead / Lider Techniczny

Decyzje architektoniczne. Code review. Mentoring juniora. 30% management, 70% coding. Kluczowy dla jakości technicznej.

UX/UI Designer

Research, wireframes, prototypy w Figma. Design system. Pracuje sprint do przodu. Niezbędny od dnia 1.

Backend Developer

API REST/GraphQL, baza danych, integracje. Mobile-first API design. Na wczesnym etapie = fullstack.

Scrum Master / Agile Coach

Facylituje ceremonie. Usuwa przeszkody (impediments). NIE zarządza zespołem – servant leader. Czas: 50-100% (zależy od dojrzałości).

iOS / Android Developer

Implementacja feature'ów. Unit testy. Uczestnictwo w code review. Senior: 5+ lat. Mid: 2-5 lat. Junior: 0-2 lata.

QA Engineer

Plan testów, przypadki testowe, automatyzacja (Appium, Detox). Definicja Done. Bug triage.

DevOps / Platform Eng.

CI/CD pipeline (GitHub Actions/Fastlane), EAS Build, monitoring, App Store deployment automation.

Inhouse (własny zespół)

Zalety

- ✓ Pełna kontrola kultury
- ✓ Głęboka znajomość produktu
- ✓ Szybka komunikacja
- ✓ IP w firmie

Wady

- ✗ Wysokie koszty rekrutacji
- ✗ Czas onboardingu 2-6 mies.
- ✗ Trudno skalować szybko
- ✗ Ryzyko rotacji

~\$8 000–25 000/mies. per developer (PL)

Software House (outsourcing)

Zalety

- ✓ Szybki start (2-4 tyg.)
- ✓ Kompletny zespół od razu
- ✓ Znany koszt miesięczny
- ✓ Elastyczne skalowanie

Wady

- ✗ Niższa znajomość domeny
- ✗ Rotacja zasobów
- ✗ Komunikacja overhead
- ✗ Mniej zaangażowania

~\$4 500–12 000/mies. per developer (PL)

Model hybrydowy

Zalety

- ✓ Inhouse core team
- ✓ Outsourcing pik'ów
- ✓ Optymalne koszty
- ✓ Transfer wiedzy

Wady

- ✗ Złożone zarządzanie
- ✗ Dwa procesy
- ✗ Ryzyko silosów
- ✗ Wymaga dojrzałości

Używany przez 58% scale-up'ów (Deloitte)

💡 Jeff Bezos: "Two-pizza rule,,, jeśli zespół nie może być nakarmiony dwoma pizzami, jest za duży. Optymalny rozmiar cross-functional team: 5–9 osób (Scrum Guide).

Metodyki pracy

Scrum, Kanban, SAFe: wybór i konfiguracja dla projektów mobilnych.

Sprint Planning

2h / 2 tyg.

PO + Dev Team. Cel sprintu + selekcja backlogu. Velocity = punkty story z poprzednich 3 sprintów.

Daily Standup

15 min / dzień

Co robiłem? Co robię? Co blokuje? Stojąc. Bez rozwiązywania problemów on-the-spot. Parking lot.

Sprint Review

1h / 2 tyg.

Demo dla stakeholders. Tylko działający software. Zbieranie feedbacku. Aktualizacja backlogu.

Retrospective

1.5h / 2 tyg.

Stop / Start / Continue. 4Ls (Liked/Learned/Lacked/Longed For). Action items z właścicielem.

Backlog Refinement

1h / tyg.

PO + Tech Lead. Story pointing (Fibonacci). Kryterium akceptacji (Given/When/Then). Dekompozycja.

Velocity mobilnego zespołu (5 dev): ~40-80 story points/sprint. Sprint 1 zawsze wolniejszy (setup CI/CD, arch. decisions). Mierz, nie estymuj godzinami – to pułapka.

Kanban board: przykład dla mobile

Backlog · WIP: ∞	In Progress · WIP: 2	Review · WIP: 1	Done · WIP: ∞
Push notifications	Auth flow	Login screen UI	Onboarding
Dark mode	API cache layer		Home screen
iPad layout			

Porównanie metodyk: wybór dla projektu

Kryterium	Scrum	Kanban	SAFe
Rozmiar zespołu	5–9 osób	1–50+ osób	50–5000 osób
Cykl releasu	Co sprint (2 tyg.)	Ciągły flow	Program Increment (PI)
Zmiany w trakcie	Tylko nowy sprint	W każdej chwili	Planowane kwartalnie
Narzut administracyjny	Średni	Niski	Wysoki

Rekomendacja: Startup/MVP → Kanban (prostota). **Zespół** 5-15 osób → Scrum. **Kilka zespołów mobilnych** → SAFe lub LeSS (Large-Scale Scrum).

Planowanie techniczne

Architektura, tech stack i roadmapa produktu.

ADR: Architecture Decision Record

Kontekst Dlaczego decyzja jest potrzebna? Jakie siły działają?

Decyzja Co zdecydowaliśmy? Jedno zdanie aktywne:
"Używamy X ponieważ Y".

Status Proposed / Accepted / Deprecated / Superseded

Konsekwencje Pozytywne (zalety) i negatywne (kompromisy).
Jawne trade-offs.

Tech stack: kluczowe decyzje architektoniczne

Native vs Cross-Platform

React Native / Flutter jeśli team < 8 i budżet < 2M PLN.
Native jeśli performance-critical lub AR/VR.

State Management

MobX (prosta apka), Redux/RTK (complex). iOS:
Combine+SwiftUI. Android: StateFlow+ViewModel.

Offline first?

SQLite / Room / Realm + sync strategy. Definiuj
conflict resolution PRZED coding.

API contract

OpenAPI 3.0 spec PRZED implementacją. Contract
testing (Pact). Consumer-driven.

Badania ThoughtWorks: zespoły które dokumentują ADR mają o 40% mniej nieplanowanych spotkań architektonicznych. ADR przechowuj w repozytorium (katalog /docs/adr).

Roadmapa: przykład 12-miesięczny (MVP → Product-Market Fit)

Faza 0 Discovery

Mies. 1-2

- ▶ Research & walidacja
- ▶ Arch. decisions
- ▶ Design sprint
- ▶ Team onboarding

Faza 1 MVP

Mies. 3-5

- ▶ Core features
- ▶ CI/CD setup
- ▶ Alpha testing
- ▶ Soft launch

Faza 2 Growth

Mies. 6-8

- ▶ User feedback loop
- ▶ Performance opt.
- ▶ A/B testy
- ▶ Push / retention

Faza 3 Scale

Mies. 9-12

- ▶ Monetyzacja
- ▶ New platforms
- ▶ Refactor debt
- ▶ Team scale-up

Struktura kosztów projektu mobilnego (typical mid-size app, rynek PL)

Development (iOS+Android)



45–55%

UX/UI Design



12–18%

Backend / API



10–15%

QA & Testing



8–12%

PM & Discovery



5–8%

DevOps / Infra



3–5%

Proces UX/UI Design

Design sprint, prototypowanie i testy użyteczności.

6.1 Design Sprint i UX Research: od problemu do przetestowanego prototypu

Pon. Understand

Map problemu, wywiady eksperckie, HMW (How Might We), cel długoterminowy.

Wt. Sketch

Crazy 8's, Lightning demos, Solution sketch. Każdy osobno.

Śr. Decide

Dot voting, Decider vote, storyboard (15 kroków na flipcharcie).

Czw. Prototype

Figma high-fidelity. Tylko to co zobaczą testerzy - nie cały flow.

Pt. Test

5 użytkowników (Jakob Nielsen). Moderowane wywiady. Think-aloud protocol.

Kluczowe metryki UX i kiedy badać

Task Completion Rate

>85%

Kiedy: Po wireframe i po launch · <70% = poważny problem nawigacyjny

Time on Task

Benchmark

Kiedy: Prototyp vs produkcja · 3x dłużej niż ekspert = zły UX

System Usability Scale (SUS)

>68 pkt

Kiedy: Beta testing · >85 = excellent. Apple produkty: ~80

App Store Rating

>4.2★

Kiedy: Po launch i per release · <3.8★ = problem retencyjny

Lider zespołu mobilnego

Obowiązki, kompetencje, techniki zarządzania i częste błędy

1 Wizja techniczna i decyzje architektoniczne

Przygotowuje i moderuje ADR. Utrzymuje architectural fitness function. Organizuje Architecture Review Board (ARB) raz na sprint. Odpowiada za tech debt roadmap.

2 Zarządzanie ryzykiem projektu

Prowadzi Risk Register (prawdopodobieństwo × wpływ). Identyfikuje single points of failure w zespole. Eskaluje blokery do PM w ciągu 24h.

3 Mentoring i rozwój zespołu

1:1 minimum 30 min/tydzień per osoba. Individual Development Plan (IDP) dla każdego. Pair programming juniora z seniorem ≥20% czasu.

4 Jakość kodu i standardy

Ustala coding guidelines (EditorConfig, linting, Detekt/SwiftLint). Pull Request template. Code review: 80% coverage minimum. Definicja Done egzekwowana.

5 Komunikacja ze stakeholders

Weekly status report (2 akapity: osiągnięcia + ryzyka). Demo per sprint. Tłumaczy complexity na język biznesowy. Buffer między tech a biznesem.

6 Planowanie zasobów i możliwości przerobowych

Uwzględnia urlopy, L4, onboarding nowych. Buffer na unplanned work: 20% sprint velocity. Nie obiecuje 100% capacity na nowe features.

7 Kultura i dobrostan zespołu

Mierzy morale zespołu (monthly mood check). Zapobiega burnoutowi: max 45h/tydzień jako norma. Celebryzuje sukcesy (retrospective appreciation round).

Struktura spotkania 1:1 (30 min / tydzień)

0–5 min

Check-in osobisty

Samopoczucie, nastrój, co poza pracą

5–15 min

Postęp i blokery

Co działa? Co nie? Gdzie potrzebujesz pomocy?

15–22 min

Feedback

Konkretny, rzeczowy. SBI: Situation-Behavior-Impact

22–28 min

Rozwój

Gdzie chcesz być za rok? Co zrobisz do następnego 1:1?

28–30 min

Action items

Max 2-3 konkretne działania z właścicielem i terminem

Macierz komunikacji projektu

Daily StandupCały team · Codziennie 9:00 · 15 min
Async-first (loom) gdy distributed**Sprint Review**Team + stakeholders · Koniec sprintu · 1h
Demo TYLKO działającego software**1:1**Lider ↔ Developer · Co tydzień · 30 min
Prywatne, poufne, focus na osobę**Risk Review**PO + Tech Lead · Co 2 tyg. · 30 min
Risk register update, escalation**All-hands**Firma · Miesięcznie · 1h
Vision, metrics, Q&A z CEO**Antywzorce lidera, czyli czego UNIKAĆ:**

- ✗ Micromanagement: "dlaczego to zajęło 3h?" – niszczy autonomię i motywację
- ✗ Brak feedbacku: milczenie ≠ akceptacja. Pracownik domyśla się błędnie
- ✗ "Heroic programmer": lider sam ratuje sprinty, a to blokuje wzrost zespołu

QA i strategia testowania

Piramida testów, automatyzacja, definicja Done i zarządzanie długiem technicznym.

Piramida testów (Mike Cohn)

E2E Tests (UI) · 10%

Integration Tests · 20%

Unit Tests · 70%

Narzędzia: Detox, Maestro, Appium

Najdroższe, wolne, kruche

Narzędzia: XCTest, Espresso, MStest

Srednie. API + DB + UI integracja

Narzędzia: Jest, JUnit, XCTest, Mockito

Szybkie <1s, tanie, deterministyczne

Definicja Done (DoD): wzorcowa dla mobile

Kod

Code review przez ≥ 1 seniora. Brak komentarzy TODO. Linting = 0 errors.

Testy

Unit test coverage $\geq 80\%$. Regression suite przeszedł. Brak P0/P1 bugów.

Wydajność

< 2s cold start. < 60MB memory footprint. Frame rate ≥ 60 fps na average device.

Dostępność

WCAG 2.1 AA. TalkBack/VoiceOver testowany. Contrast ratio $\geq 4.5:1$.

Dokumentacja

README zaktualizowane. API changelog. Release notes draft.

💡 **Code review** redukuje liczbę bugów o 60% (SmartBear, 2023). Każdy bug znaleziony w fazie testów kosztuje 10x więcej do naprawienia po launchu (NIST).

23%**czasu dev**

spędzone na tech debt (McKinsey 2023)

40%**wolniejszy**

development przy wysokim tech debt

\$1.52T**globalny koszt**

technicznego długu (CISQ 2023)

20%**sprint**

na spłatę długu (rekomendacja)

Rodzaje długu technicznego i strategie spłaty

Deliberate (świadomy)

"Zrobimy to szybko teraz, naprawimy później." Akceptowalny jeśli jest decyzja świadoma i udokumentowana w ADR.

Action: Ticket w backlogu z datą spłaty. Nie starszy niż 2 sprinty.

Inadvertent (nieświadomy)

"Nie wiedzieliśmy że tak nie powinno się robić." Pojawia się przy wzroście know-how lub zmianie standardów.

Action: Coding guidelines + regular Tech Debt Sprint co Q.

Rotten Code (degradacja)

Biblioteka nieaktualizowana od 2 lat. Deprecated API. Kod który "nikt nie dotyka bo boi się". Timebomb.

Action: Dependency audit co sprint. Automatyczne Dependabot/Renovate.

Test Debt

Brak testów, flaky testy ignorowane, manual-only regression. Każdy deploy to roulette.

Action: Coverage gate w CI (SonarQube). Brak merge jeśli coverage spada.

Launch, launch i po-launchu

App Store submission, metryki sukcesu, zarządzanie kryzysem i iteracja.

Checklist przed submitacją (App Store Connect / Google Play Console)

☐ **App Store Screenshots**

6.7" + 6.5" + 12.9" + 5.5". Tekst marketingowy na grafice.

☐ **Keywords (ASO)**

Max 100 znaków. Nie powtarzaj nazwy. Używaj SensorTower.

☐ **Crash rate < 1%**

TestFlight beta ≥ 500 users, 7 dni bez P0 bugów.

☐ **Performance baseline**

< 3s cold start. < 200ms API latency p95.

☐ **Rollback procedure**

Przygotuj poprzedni build. Max 1h do rollback decision.

☐ **App Preview Video**

15-30s. Pierwsze 3 sekundy krytyczne. Bez lektora.

☐ **Privacy Policy URL**

HTTPS. Wymagana dla apps z logowaniem lub analityką.

☐ **Accessibility audit**

VoiceOver / TalkBack. Dynamic Type. High Contrast.

☐ **Staged rollout plan**

1% → 5% → 20% → 100% w 3 tygodniach (Android).

☐ **Monitoring & alerts**

Crashlytics, Firebase Perf, custom alerts w PagerDuty.

ASO Stat: 65% pobrań zaczyna się od wyszukiwania w App Store (Apple).

Aplikacje z video preview konwertują o 35% lepiej. Ocena 4.0→4.5★ = +89% więcej pobrań.

A

Acquisition

CPI (Cost per Install)

Target: iOS: <\$2, Android: <\$0.8

App Store Organic + Paid UA (Meta/TikTok Ads).
Źródła: Search, Browse, Referral

A

Activation

Day-1 Retention

Target: >40%

Czy użytkownik wrócił dzień po instalacji?
Onboarding flow kluczowy.

R

Retention

Day-7 / Day-30

Target: D7>20%, D30>10%

Industry avg: D7=13%, D30=6%. Top apps:
D7=50%+. Cohort analysis w Amplitude.

R

Revenue

ARPU / LTV

Target: LTV > 3× CPI

ARPU = MRR / MAU. LTV = ARPU × średni czas życia
użytkownika.

R

Referral

Viral Coefficient (K)

Target: K > 0.5

K = Invites × Conversion rate. K>1 = viral growth
(rzadkie). NPS > 50 = excellent.

North Star Metric = jedna liczba która wyraża wartość produktu dla użytkownika. **Spotify**: minuty odsłuchu. **Airbnb**: zarezerwowane noce. **WhatsApp**: wiadomości wysłane/dzień.

1h**czas do rollback**

decyzja musi zapaść w ciągu 1h

\$5 600**utrata przychodu**

na minutę downtime (avg e-commerce)

3★**porzucenie apki**

użytkownicy odinstalowują przy ocenie <3★

72h**okno krytyczne**

wizerunek można uratować w 72h od incydentu

Klasyfikacja incydentów i czas reakcji

P0 – Critical

Apka nie startuje / crash rate >10% / utrata danych użytkowników

Reakcja: 15 min**Rozwiązanie: 2h****P1 – High**

Kluczowy flow nie działa (logowanie, płatności) / degradacja >50% userów

Reakcja: 1h**Rozwiązanie: 8h****P2 – Medium**

Feature działa z błędami / błąd dla <10% userów / brak danych statystycznych

Reakcja: 4h**Rozwiązanie: 2 dni****P3 – Low**

Drobny błąd UI / literówka / feature nieużywany

Reakcja: 24h**Rozwiązanie: sprint****Post-mortem (blameless):**

Co się stało? → Dlaczego? (5×Why) → Co robimy żeby się nie powtórzyło? → Action items z właścicielem i terminem.
Kultura blameless = ludzie raportują incydenty wcześniej.

OKR: Objectives & Key Results dla projektu mobilnego

O: Stać się wiodącą aplikacją fitness w Polsce do Q4 2026 (mierzone: Top 3 w kategorii Health)

KR1	MAU wzrośnie z 50k do 200k do 31.12.2026	Amplitude DAU/MAU ratio
KR2	Day-30 retention wzrośnie z 12% do 25%	Cohort analysis w Mixpanel
KR3	App Store rating wzrośnie do 4.5★ (min. 5000 reviews)	App Store Connect
KR4	MRR z subskrypcji osiągnie 150 000 PLN	RevenueCat dashboard

Pętle feedbackowe: ciągła poprawa produktu

Dziennie

Źródło: Crash reports, App Store reviews, support tickets
→ Hotfix jeśli P0/P1, triaging reszty

Co sprint

Źródło: Sprint review demo, Analytics dashboard, A/B test wyniki
→ Adjustment backlogu, re-prioritization

Miesięcznie

Źródło: NPS survey, user interviews (5 os.), churn analysis
→ Roadmap update, feature kill/keep/build

Kwartalnie

Źródło: OKR review, market benchmark, competitor analysis
→ Strategia produktu, tech investment plan

💡 **Amazon:** "Every day is launch day" – ciągłe wydawanie małych zmian jest bezpieczniejsze niż wielki release co 6 miesięcy. **Netflix** wypuszcza setki deploymentów dziennie.

Podsumowanie: kluczowe zasady skutecznego projektu mobilnego

1 Waliduj zanim zakodujesz

Każdy tydzień walidacji oszczędza miesiąc developmentu. 42% projektów upada z powodu braku potrzeby rynkowej.

2 Planowanie to inwestycja

Discovery + ADR + roadmapa = mniej zaskoczeń.
Brak planu = plan na niepowodzenie.

3 Ludzie ponad procesy

Najlepszy stack technologiczny nie zastąpi zmotywowanego zespołu z jasną wizją i dobrym liderem.

4 Mierz wszystko

AARRR, OKR, DORA metrics (Deployment Frequency, Lead Time, MTTR, Change Failure Rate).

5 Iteruj szybko, myśl długo

Małe częste releasy, duże strategiczne decyzje rzadziej. Build-Measure-Learn loop.

6 Dług techniczny = hipoteka

Splacaj regularnie 20% sprintu. Zaniedbany rośnie z odsetkami i spowalnia cały projekt.

"Plans are worthless, but planning is everything." - Dwight D. Eisenhower · Programowanie urządzeń mobilnych wymaga tyle samo inżynierii organizacyjnej, co technicznej.